분야 AI/SW 심화 구분 자연어 처리 학습시간 70시간 필수/선택 필수

원하는 내용이 들어있는 문서를 똑똑하게 찾아주는 검색기 만들기

문제기술

기술적 설명



1. 미션 소개

자연어 처리(NLP)는 검색 엔진, 추천 시스템, 스팸 필터 등 텍스트 기반 서비스의 핵심 기술이다. 이 미션에서는 뉴스 기사 또는 리뷰 데이터를 활용하여 문서 검색 및 분류 시스템을 구축하며, 텍스트를 숫자 벡터로 변환하는 전체 과정(토큰화, TF-IDF 벡터화, 유사도 계산)을 직접 구현한다. 이 과정을 통해 "단어의 출현 빈도만으로 문서의 의미를 파악할 수 있는가?"라는 질문에 답하며, BoW(Bag of Words) 방식의 가능성과 한계를 직접 체감한다.

2. 최종 결과물

다음 내용이 모두 포함된 GitHub 레포지토리(Repository) URL을 제출한다.

1. 전처리 및 TF-IDF 구현 코드
    - 토큰화, 불용어 제거 파이프라인
    - NumPy를 활용한 TF-IDF 직접 구현
    - Scikit-learn 결과와의 검증 로그 (사용한 sklearn 설정 명시)
2. 문서 검색 및 분류 시스템
    - 코사인 유사도 기반 검색 기능
    - TF-IDF 벡터 기반 분류 모델 및 성능 평가
3. 분석 리포트 (README.md) - 필수 3단 구성

- 구현 요약: 전처리 파이프라인, TF-IDF 수식과 구현 과정, 검색/분류 시스템
- 검증/실험 결과: TF-IDF 검증 로그, 분류 성능 지표, 혼동 행렬
- 한계와 개선 방향: BoW 한계 분석, 오분류 원인, 후속 학습 방향

4. 실행 방법 명시 (필수)

- README에 아래 중 하나 이상의 실행 방법을 반드시 명시한다.
    - (안 A) CLI: python main.py --query "..." --topk 5 형태의 실행 커맨드와 예시 출력
    - (안 B) 함수: search(query: str) -> List[(score, doc_id, text_snippet)] 형태의 시그니처

3. 과제 목표

이 과제를 마친 후, 학습자는 아래를 스스로 설명할 수 있어야 한다.

- 토큰화(Tokenization)의 목적과 언어별 처리 방식의 차이를 설명할 수 있다.
- TF, IDF, TF-IDF의 수학적 정의를 수식으로 표현하고 각각의 역할을 설명할 수 있다.
- 코사인 유사도가 벡터의 내적과 정규화로 계산되는 원리를 이해한다.
- BoW 방식이 문맥과 어순을 반영하지 못하는 한계를 구체적 사례와 함께 설명할 수 있다.
- 전처리 기준(불용어, 특수문자 범위)을 설계한 근거를 설명할 수 있다.

4. 기능 요구 사항

다음 요구사항을 모두 만족해야 한다.

1. 데이터 준비 및 전처리

- 최소 500개 이상의 문서가 포함된 텍스트 데이터셋을 사용한다.
- 텍스트 정제(특수문자 제거, 소문자 변환 등)를 수행한다.
- 토큰화를 구현한다. (한국어: 형태소 분석기 권장, 영어: NLTK 또는 공백 기반)
    - 한국어 형태소 분석기 설치가 어려운 환경의 경우, 공백 토큰화를 대안으로 사용하되 "성능이 왜 달라지는지"를 리포트에 분석한다.
- 불용어 목록을 정의하고 제거한다.
- 전처리 기준(특수문자 제거 범위, 불용어/동의어 처리 기준)을 학습자가 직접 설계하고 근거를 리포트에 서술한다.

2. TF-IDF 직접 구현

- NumPy를 사용하여 TF-IDF 행렬을 계산하는 함수를 작성한다.
- TF 계산: 문서 내 단어 빈도를 행렬로 표현한다.
- IDF 계산: 전체 문서 대비 단어 출현 문서 수의 역수(로그 적용)를 계산한다.
- 구현 결과를 Scikit-learn TfidfVectorizer와 비교하여 오차가 1e-6 이내인지 검증한다.
    - 검증 재현성 확보: sklearn의 sublinear_tf, smooth_idf, norm 등의 파라미터를 직접 구현체와 동일하게 설정하고, 어떤 설정으로 검증했는지 README에 명시한다.

3. 문서 검색 시스템

- 쿼리를 TF-IDF 벡터로 변환한다.
- 코사인 유사도를 NumPy 벡터 연산으로 직접 계산한다.
- 유사도 상위 5개 문서를 반환한다. (점수, 문서 ID, 텍스트 스니펫 포함)

4. 문서 분류 실험

- TF-IDF 벡터를 특성으로 사용하여 분류 모델을 학습한다.
- 기본선: Logistic Regression 또는 Linear SVM 중 택1, Train/Test 8:2, random_state 고정
- 정확도, F1-Score를 측정하고 혼동 행렬을 시각화한다.
- 오분류된 문서 5개 이상의 원인을 분석한다. (BoW 한계와 연결하여 서술)

5. 보너스 과제

1. 다양한 TF 변형 구현

- Raw Count, Log Normalization, Double Normalization 등 TF 계산 방식을 비교 구현한다.
- 각 방식이 검색/분류 성능에 미치는 영향을 정량적으로 분석한다.

2. BM25 구현

- BM25의 수식을 이해하고 NumPy로 직접 구현한다.
- 동일한 쿼리에서 TF-IDF와 BM25 검색 결과를 비교 분석한다.

3. 역색인(Inverted Index) 구현

- 단어 → 문서 ID 리스트 형태의 역색인 구조를 구현한다.
- 역색인 기반 검색과 전체 스캔 검색의 시간 복잡도를 비교한다.

[참고 사항] 데이터셋 확보 가이드

1. 한국어 데이터셋

   [추천 1] 네이버 영화 리뷰 (NSMC)

   - 설명: 네이버 영화 리뷰 20만 건 (긍정/부정 레이블)
   - 다운로드: github.com/e9t/nsmc 또는 Hugging Face datasets

```
CSS
# NSMC 로드 예시
import pandas as pd
train = pd.read_csv('ratings_train.txt', sep='    ')
# 또는 Hugging Face
from datasets import load_dataset
dataset = load_dataset('nsmc')
```

   [추천 2] AI Hub 한국어 뉴스 데이터

   - 설명: 카테고리별 뉴스 기사 (정치, 경제, 사회, 문화, IT 등)
   - 다운로드: aihub.or.kr (회원가입 필요)

2. 영어 데이터셋

   [추천 1] 20 Newsgroups

   - 설명: 20개 카테고리 뉴스그룹 게시물 약 18,000건
   - 장점: Scikit-learn 내장, 즉시 사용 가능

```
CSS
# 20 Newsgroups 로드 예시
from sklearn.datasets import fetch_20newsgroups
categories = ['sci.space', 'comp.graphics', 'rec.sport.baseball']
data = fetch_20newsgroups(subset='all', categories=categories)
print(f'문서 수: {len(data.data)}, 카테고리: {data.target_names}')
```

   [추천 2] AG News / IMDB

   - AG News: 4개 카테고리 뉴스 기사
   - IMDB: 50,000개 영화 리뷰 (긍정/부정)

- 다운로드: Hugging Face datasets

3. 데이터 준비 체크리스트

- 500개 이상 문서 확보
- 2개 이상 카테고리 포함
- Train/Test 분리 가능
- 결측치 제거 완료
- 개인정보 미포함 확인
- 라이선스/저작권 확인 완료

개발환경

6. 개발 환경

- Python 3.10 이상

제약조건

7. 제약 사항

- TF-IDF 구현 : 핵심 로직은 NumPy만 사용한다. Scikit-learn은 검증 용도로만 사용한다.
- 코사인 유사도 : NumPy 벡터 연산으로 직접 구현한다.
- 허용 라이브러리 : Pandas, NumPy, Matplotlib/Seaborn, KoNLPy(한국어), NLTK(영어), Scikit-learn(분류/검증용)
- 데이터 : 최소 2개 이상 카테고리를 포함하는 분류 문제를 다룬다.
- 실행 환경 : Python 3.10+ 권장. requirements.txt 제출 필수.
- 데이터 윤리 : 개인정보(식별 가능한 이름, 연락처, 건강정보 등) 포함 데이터 사용 금지. 공개 데이터셋 우선 사용.

Test Case

8. 결과 예시

아래는 정답이 아니라 참고 예시다. 실제 문구와 구조는 달라도 된다.

- TF-IDF 직접 구현 검증 :

```
CSS
=== TF-IDF 구현 검증 결과 ===
TF-IDF 행렬 shape: (500, 3247)
[검증] 직접 구현 vs Scikit-learn
  - sklearn 설정: smooth_idf=True, sublinear_tf=False, norm='l2'
  - 최대 오차: 8.32e-10
  - 평균 오차: 1.05e-12
  - 결과: PASS (허용 오차 1e-6 이내)
```

- 코사인 유사도 계산 과정 :

```
CSS
=== 코사인 유사도 계산 예시 ===
Query: "딥러닝 이미지 분류"

Query 벡터: [0.0, 0.52, 0.0, 0.31, ...]
문서 벡터:  [0.12, 0.48, 0.0, 0.29, ...]

코사인 유사도 = (Query · 문서) / (||Query|| × ||문서||)
              = 0.347 / (0.61 × 0.58) = 0.981
```

- 문서 검색 결과 :

```
CSS
=== 검색 결과 ===
Query: "딥러닝 이미지 분류"

Top 5 결과:
1. [0.847] "CNN 기반 이미지 분류 모델 구현..."
2. [0.723] "딥러닝을 활용한 객체 탐지..."
3. [0.698] "이미지 인식 기술의 발전..."
4. [0.612] "컴퓨터 비전 최신 동향..."
5. [0.589] "신경망 구조 비교 분석..."
```

- 분류 모델 성능 :

```
CSS
=== 분류 성능 평가 ===
모델: Logistic Regression
데이터: Train 400 / Test 100

정확도: 0.847
F1-Score (macro): 0.832
```

- 오분류 분석 :

```
CSS
=== 오분류 케이스 분석 ===

[케이스 1]
예측: 스포츠 / 실제: IT
문서: "애플이 새로운 골프 앱을 출시했다"
원인: '골프' 단어의 TF-IDF가 스포츠에서 높게 학습됨
한계: 단어의 문맥(애플=회사)을 반영하지 못함
```

- BoW 한계 분석 :

```
CSS
=== BoW 한계 분석 ===

[문제 1: 어순 무시]
"고양이가 쥐를 잡았다" vs "쥐가 고양이를 잡았다"
→ 동일한 TF-IDF 벡터 생성

[문제 2: 동음이의어]
"배가 고프다" vs "배를 타다" vs "배 과수원"
→ '배' 단어가 모두 동일하게 처리됨
```